

DOCUMENTATION TECHNIQUE

Projet 2 : Centralisation du monitoring et gestion de configuration d'une infrastructure hybride (Proxmox, MikroTik & Cloud)

Candidat : DENELLE Mahoé

BTS Services Informatiques aux Organisations

Option : Solutions d'Infrastructure, Systèmes et Réseaux (SISR)

Établissement : CFA SDMI

Session : 2026

Organisation support : DC2Scale

Technologies clés :

Hyperviseur : Proxmox VE 9.1

Supervision : Zabbix & Grafana

Réseau : MikroTik (Tunnel EoIP & SNMP)

Backup : Oxidized



I. Introduction	3
II. Réalisation Technique : Supervision et Gestion des Équipements	4
1. Supervision (Serveur Ubuntu 24.04 - 82.165.174.23)	4
A - Installation de Zabbix 7.0	4
B - Installation de Grafana	5
C - Synchronisation Zabbix et Grafana	6
2. Hyperviseur (Proxmox VE 9.1 - 213.5.130.126)	7
A - Configuration Réseau et Sécurité (Host)	7
B - Installation de l'Agent Zabbix sur Proxmox	8
3. Infrastructure Réseau MikroTik (Tunnel EoIP)	8
A - Routeur Local (VM sur Proxmox - 192.168.100.2)	8
B - Routeur Distant (VM OVH - 151.247.192.139)	9
C - Activation SNMP pour Zabbix	9
4. Ajout et configuration des hôtes (Hosts) dans Zabbix	10
1. Ajout de l'hyperviseur Proxmox (via Agent)	10
2. Ajout du Routeur Distant OVH (via SNMP)	11
3. Ajout du Routeur Local (via SNMP)	11
5. Gestion de Configuration (Oxidized - 151.247.192.142)	12
A - Installation sur Ubuntu 24.04	12
B - Fichier de Configuration Oxidized	12
6. Alerting et Dashboards	14
A - Création du Dashboard	14
1. Panel de l'Hyperviseur Proxmox (Agent Zabbix)	14
2. Panel du Tunnel EoIP (SNMP)	14
B - Configuration des Alertes Critiques et Triggers	15
1. Alerte de rupture du Tunnel (Flux réseau)	15
2. Alerte thermique (Sécurité matérielle)	15
C - Notification via Contact Points (Bot Telegram)	16
III. Conclusion	17

I. Introduction

Le projet présenté prend place au sein du contexte de la société **DC2Scale**. Dans le but d'apporter une visibilité totale sur l'infrastructure et de garantir la continuité de service, je dois mettre en place diverses solutions :

- Une instance de supervision sous Debian 12 pour centraliser les métriques ;
- Un monitoring profond de l'hyperviseur pour surveiller la santé du serveur ;
- Une supervision réseau via SNMP pour analyser le trafic des routeurs ;
- Une solution d'archivage automatique des configurations réseaux (Oxidized).

L'environnement technologique se compose du matériel suivant :

- Un hyperviseur **Proxmox VE 9.1** (Serveur physique) ;
- Un routeur distant (VM Mikrotik) sur un serveur **OVHCloud** ;
- Un routeur local (VM MikroTik) sur l'hyperviseur ;
- Un **tunnel EoIP** entre le routeur local et distant ;
- Une machine virtuelle sous Ubuntu 24.04 (Serveur Zabbix / Grafana) ;
- Une machine virtuelle sous Ubuntu 24.04 (Serveur Oxidized)

Le serveur de supervision est configuré en IP statique et interroge les équipements à travers le tunnel de niveau 2. Les agents Zabbix permettent une remontée de données précise sur l'état du système, tandis que le protocole SNMP est utilisé pour les équipements réseaux.

L'utilisation de Grafana permet de transformer ces données en tableaux de bord visuels pour faciliter l'aide à la décision. Le système d'alerting garantit quant à lui une réactivité immédiate en cas de rupture du lien entre les deux sites.

II. Réalisation Technique : Supervision et Gestion des Équipements

1. Supervision (Serveur Ubuntu 24.04 - 82.165.174.23)

A - Installation de Zabbix 7.0

L'installation repose sur une pile LAMP (Linux, Apache, MariaDB, PHP).

1. Configuration du dépôt : Connexion SSH au serveur Ubuntu.

```
wget
https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_7.0-1+ubuntu24.04_all.d
eb
dpkg -i zabbix-release_7.0-1+ubuntu24.04_all.deb
apt update
```

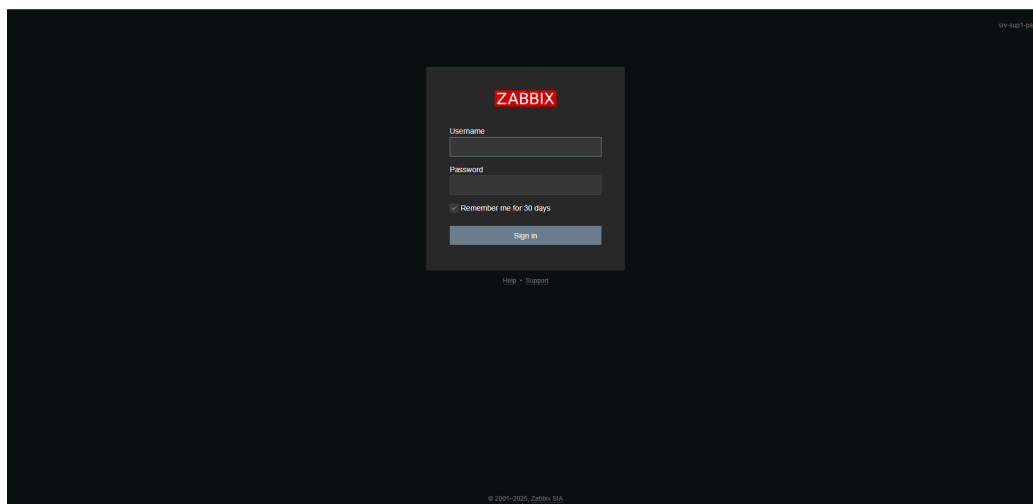
2. Installation des paquets :

```
apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-conf zabbix-sql-scripts zabbix-agent
mariadb-server -y
```

3. Base de données : Initialisation de MariaDB et création de l'utilisateur zabbix.

```
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin; CREATE
USER 'zabbix'@'localhost' IDENTIFIED BY 'TonMotDePasseSecurise'; GRANT ALL
PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost'; SET GLOBAL
log_bin_trust_function_creators = 1; EXIT;
```

4. Finalisation Web : Accès via l'URL <http://82.165.174.23/zabbix>.



[Figure 1 : Capture de la page login Zabbix après la configuration]

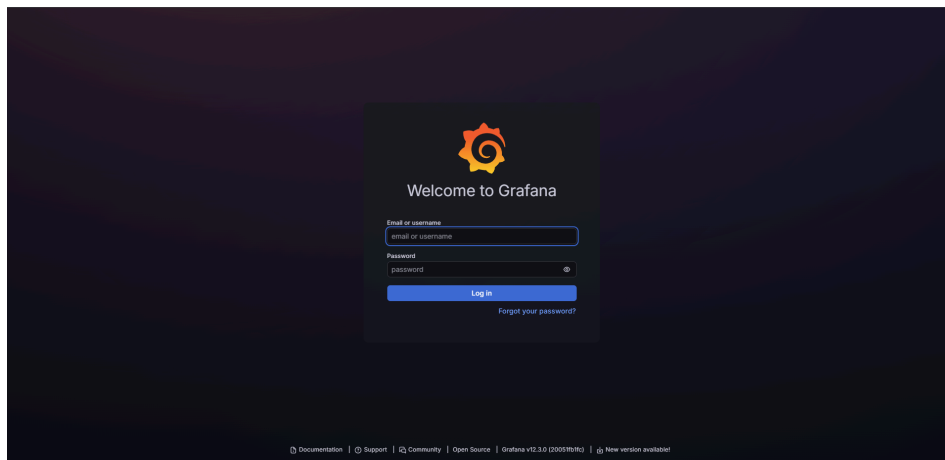
B - Installation de Grafana

1. Installation via dépôt officiel :

```
apt-get install -y apt-transport-https software-properties-common wget  
mkdir -p /etc/apt/keyrings/  
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor > /etc/apt/keyrings/grafana.gpg  
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | tee  
/etc/apt/keyrings/grafana.list  
apt update && apt install grafana -y
```

2. **Démarrage** : `systemctl enable --now grafana-server.`

3. **Accès** : `http://82.165.174.23:3000.`



[Figure 2 : Capture de l'écran de login Grafana (admin/admin)]

C - Synchronisation Zabbix et Grafana

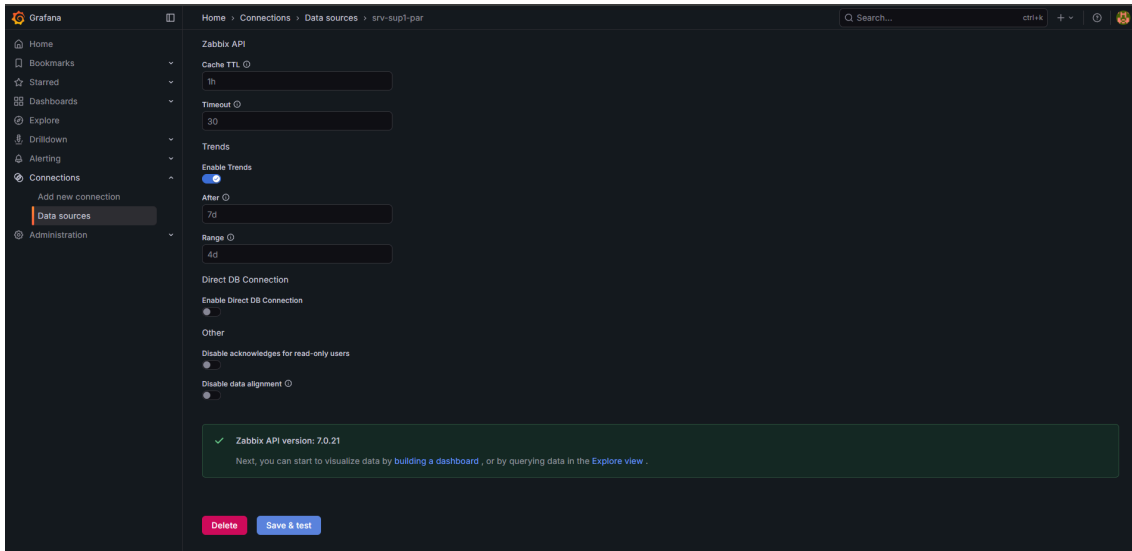
1. Installation du plugin :

```
grafana-cli plugins install alexanderzobninin-zabbix-app  
systemctl restart grafana-server
```

2. Liaison API : Dans l'interface Grafana, ajouter la source de données "Zabbix".

URL : `http://localhost/zabbix/api_jsonrpc.php`

Identifiants : Admin / Zabbix



[Figure 3 : Capture du message vert confirmant la réussite de la synchronisation]

2. Hyperviseur (Proxmox VE 9.1 - 213.5.130.126)

A - Configuration Réseau et Sécurité (Host)

Le serveur utilise une seule IP publique. Nous avons dû créer un pont (vmbri1) et gérer le routage via IPTables.

1. Fichier de configuration réseau :

Contenu du fichier `/etc/network/interfaces` :

```
auto vmbri0
iface vmbri0 inet static
    address 213.5.130.126/24
    gateway 213.5.130.1
    bridge-ports nic1

auto vmbri1
iface vmbri1 inet manual
    bridge-vlan-aware yes
```

2. Règles de translation (NAT/Forwarding) :

```
iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT

iptables -A FORWARD -s 192.168.100.0/24 -j ACCEPT

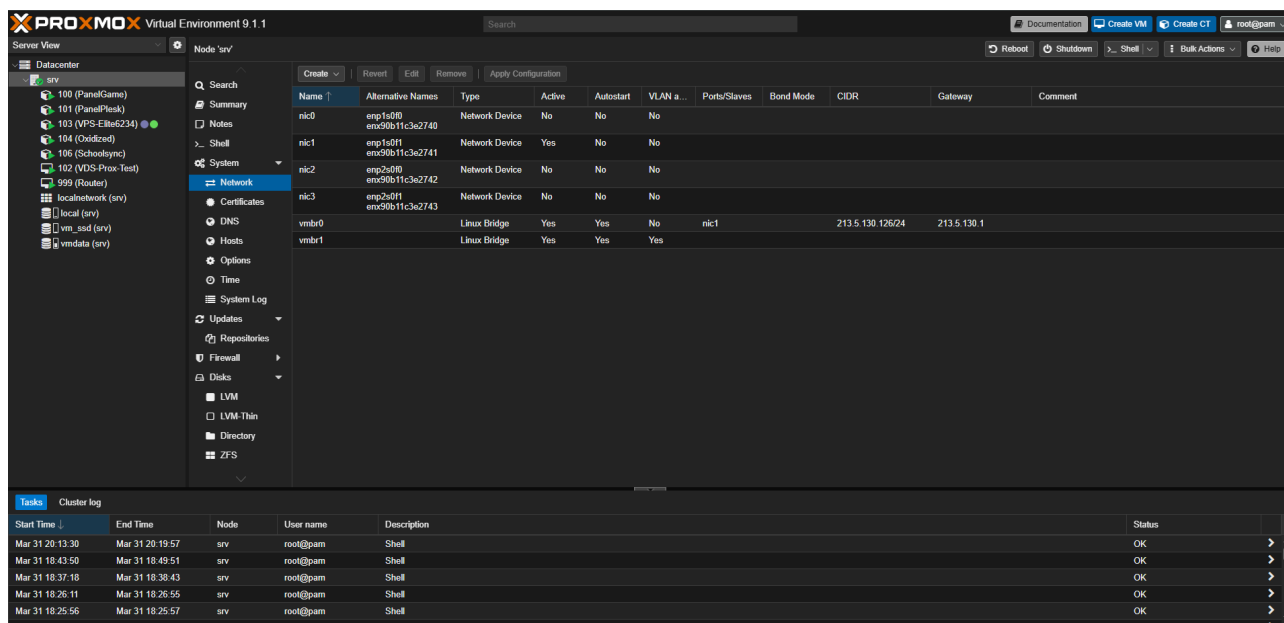
iptables -A FORWARD -d 192.168.100.0/24 -j ACCEPT

iptables -A FORWARD -p tcp -d 192.168.100.2 --dport 8291 -j ACCEPT

iptables -A FORWARD -p tcp -d 192.168.100.2 --dport 22 -j ACCEPT

iptables -A FORWARD -p gre -d 192.168.100.2 -j ACCEPT

iptables -A FORWARD -p udp -d 192.168.100.2 --dport 161 -j ACCEPT
```



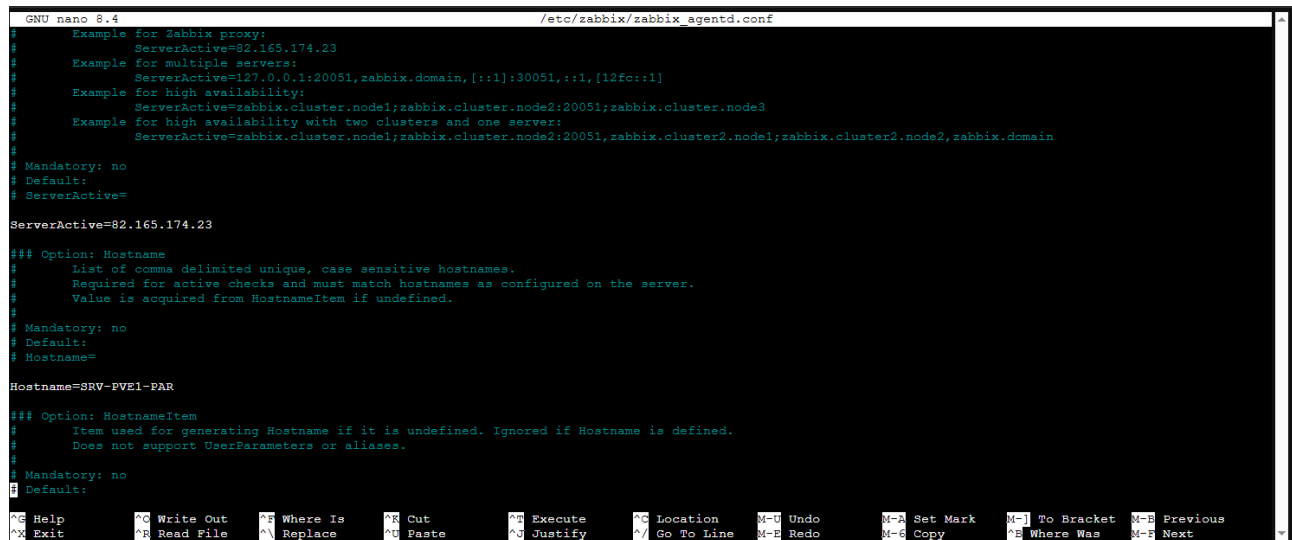
[Figure 4 : Capture d'écran du menu Network dans l'interface Proxmox]

B - Installation de l'Agent Zabbix sur Proxmox

1. **Installation** : `apt install zabbix-agent2 -y.`
2. **Configuration** : Modifier `/etc/zabbix/zabbix_agent2.conf`.

`Server=82.165.174.23`

`Hostname=SRV-PVE1-PAR`



```
GNU nano 8.4 /etc/zabbix/zabbix_agentd.conf
# Example for Zabbix proxy:
# ServerActive=82.165.174.23
# Example for multiple servers:
# ServerActive=127.0.0.1;20051,zabbix.domain,[::1]:30051,[::1],[12fc::1]
# Example for high availability:
# ServerActive=zabbix.cluster.node1;zabbix.cluster.node2;20051;zabbix.cluster.node3
# Example for high availability with two clusters and one server:
# ServerActive=zabbix.cluster.node1;zabbix.cluster.node2;20051,zabbix.cluster2.node1;zabbix.cluster2.node2,zabbix.domain
#
# Mandatory: no
# Default:
# ServerActive=
ServerActive=82.165.174.23
## Option: Hostname
# List of comma delimited unique, case sensitive hostnames.
# Required for active checks and must match hostnames as configured on the server.
# Value is acquired from HostnameItem if undefined.
#
# Mandatory: no
# Default:
# Hostname=
Hostname=SRV-PVE1-PAR
## Option: HostnameItem
# Item used for generating Hostname if it is undefined. Ignored if Hostname is defined.
# Does not support UserParameters or aliases.
#
# Mandatory: no
# Default:
```

[Figure 5 : Capture du fichier de conf montrant l'IP du serveur de supervision]

3. Infrastructure Réseau MikroTik (Tunnel EoIP)

A - Routeur Local (VM sur Proxmox - 192.168.100.2)

Ce routeur reçoit le flux via le pont interne du Proxmox.

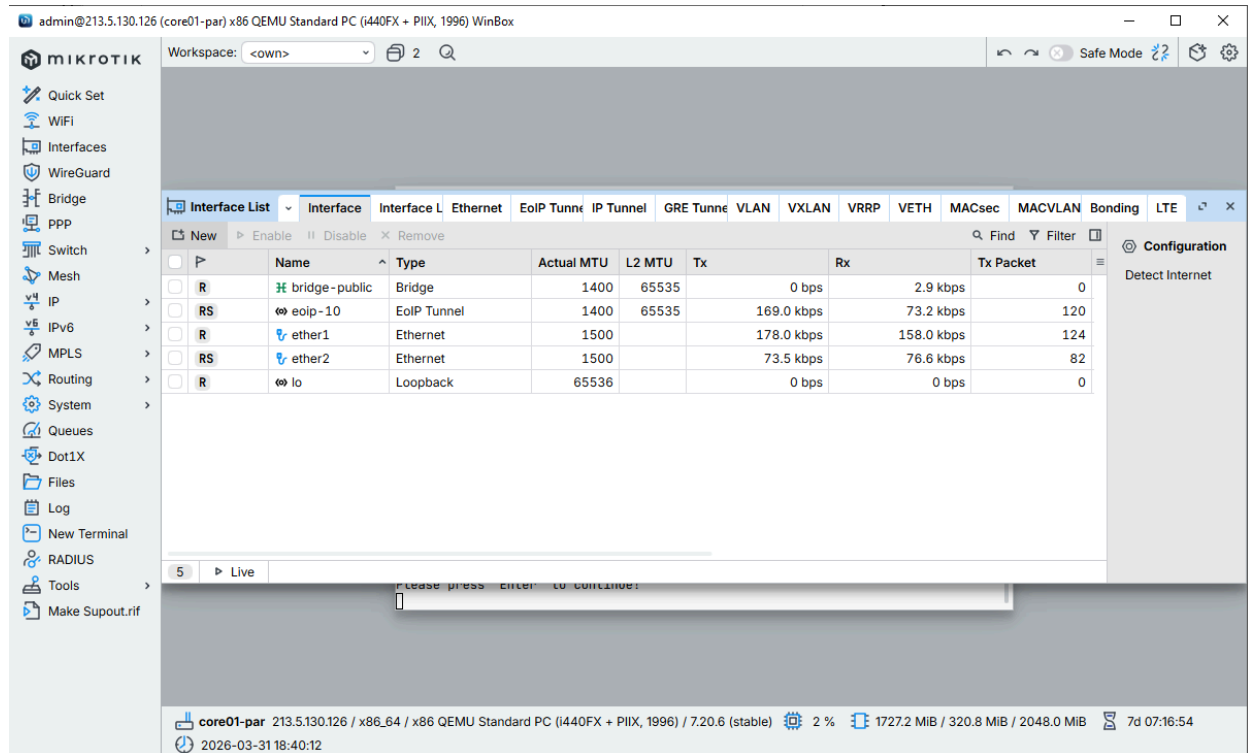
1. **Accès Winbox** : Connexion sur le port 8291 (autorisé par IPTables).
2. **Configuration EoIP** :

```
/interface eoip add name=eoip-ovh remote-address=151.247.192.139 tunnel-id=10
/interface bridge port add bridge=bridge-local interface=eoip-ovh
```

B - Routeur Distant (VM OVH - 151.247.192.139)

1. Configuration EoIP : Symétrique au routeur local.

/interface eoip add name=eoip-local remote-address=213.5.130.126 tunnel-id=10

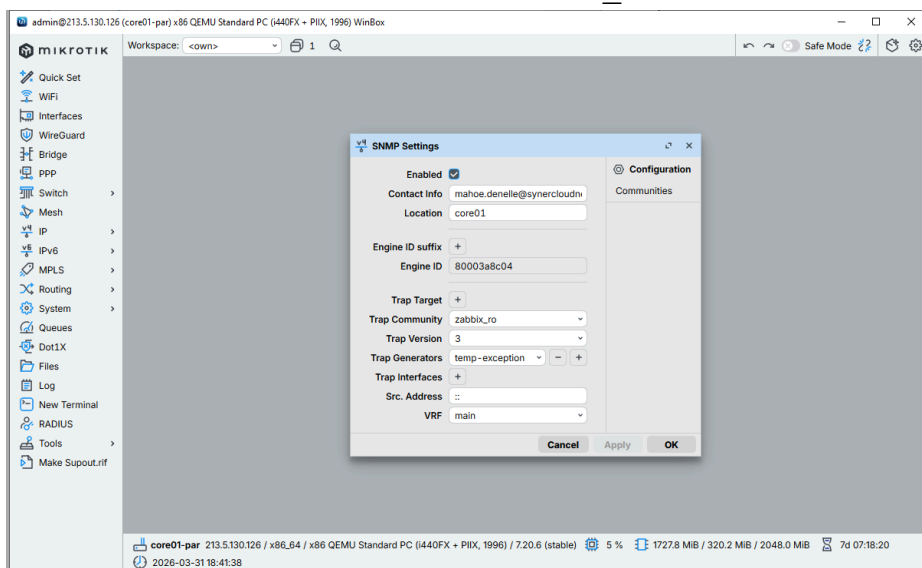


[Figure 6 : Capture de l'interface Winbox montrant le statut "R" (Running) du tunnel EoIP]

C - Activation SNMP pour Zabbix

Sur les deux MikroTik :

1. IP > SNMP > Enabled: Yes.
2. Communities > + > Name: zabbix_ro.



[Figure 7 : Capture de l'interface Winbox montrant la configuration SNMP]

4. Ajout et configuration des hôtes (Hosts) dans Zabbix

1. Ajout de l'hyperviseur Proxmox (via Agent)

1. Connectez-vous à l'interface Web : `http://82.165.174.23/zabbix`.
2. Allez dans le menu **Data Collection** > **Hosts**, puis cliquez sur le bouton bleu **Create host** en haut à droite.

3. Onglet Host :

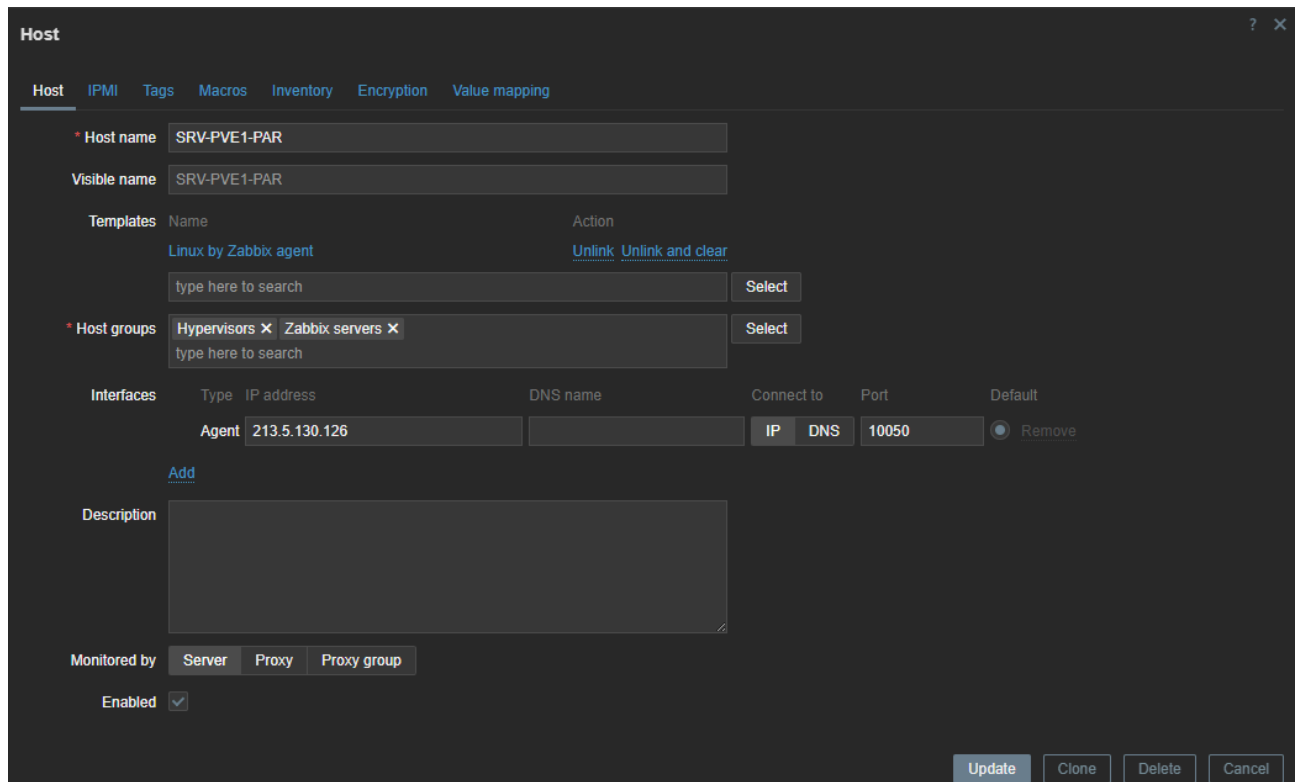
Host name : SRV-PVE1-PAR (doit être identique au Hostname dans le fichier `.conf` de l'agent).

Templates : Cherchez et sélectionnez `Linux by Zabbix agent`.

Host groups : Créez ou sélectionnez un groupe (ex: Hypervisors).

Interfaces : Cliquez sur **Add** > **Agent**. Entrez l'IP : `213.5.130.126`.

4. Cliquez sur **Add**.



The screenshot shows the 'Host' configuration form in Zabbix. The 'Host name' field is set to 'SRV-PVE1-PAR'. The 'Visible name' field is also set to 'SRV-PVE1-PAR'. Under 'Templates', 'Linux by Zabbix agent' is selected. Under 'Host groups', 'Hypervisors' and 'Zabbix servers' are selected. In the 'Interfaces' section, an 'Agent' interface is configured with IP address '213.5.130.126', 'IP' as the connection type, and port '10050'. The 'Description' field is empty. The 'Monitored by' section shows 'Server' selected. The 'Enabled' checkbox is checked. At the bottom right, there are buttons for 'Update', 'Clone', 'Delete', and 'Cancel'.

[Figure 8 : Capture d'écran du formulaire de création de l'hôte PVE]

2. Ajout du Routeur Distant OVH (via SNMP)

1. Dans **Data Collection > Hosts**, cliquez sur **Create host**.

2. Onglet Host :

Host name : CORE01-OVH.

Templates : Sélectionnez MikroTik by SNMP.

Interfaces : Cliquez sur **Add > SNMP**. Entrez l'IP : 151.247.192.139.

3. Onglet Macros :

Cliquez sur **Inherited and host macros**.

Cherchez la macro `{ $SNMP_COMMUNITY }` et modifiez la valeur par `zabbix_ro` (celle configurée sur le MikroTik).

4. Cliquez sur **Add**.

3. Ajout du Routeur Local (via SNMP)

1. Même procédure que pour le routeur distant.

2. **IP de l'interface SNMP** : 192.168.100.2.

Name	Items	Triggers	Graphs	Discovery	Web	Interface	Proxy	Templates	Status	Availability	Agent encryption	Info	Tags
CORE01-OVH	Items 67	Triggers 33	Graphs 11	Discovery 8	Web	151.247.192.139/161		Mikrotik by SNMP	Enabled	SNMP	None		
core01-par	Items 46	Triggers 32	Graphs 10	Discovery 8	Web	213.5.130.126/161		Mikrotik by SNMP	Enabled	SNMP	None		
SRV-PVE1-PAR	Items 493	Triggers 224	Graphs 56	Discovery 6	Web	213.5.130.126/10050		Linux by Zabbix agent, Zabbix server health	Enabled	ZBX	None		
SRV-SUP1-PAR	Items 153	Triggers 83	Graphs 16	Discovery 6	Web	127.0.0.1/10050		Linux by Zabbix agent, Zabbix server health	Enabled	ZBX	None		

Displaying 4 of 4 found

[Figure 9 : Capture de la liste finale des hôtes avec les icônes "ZBX" et "SNMP" toutes au vert]

5. Gestion de Configuration (Oxidized - 151.247.192.142)

A - Installation sur Ubuntu 24.04

1. **Dépendances :** `apt install ruby ruby-dev libsqlite3-dev libssl-dev pkg-config cmake -y.`
2. **Installation :** `gem install oxidized oxidized-web.`

B - Fichier de Configuration Oxidized

Contenu du fichier `/root/.config/oxidized/config` :

```
username: admin
password: Rgheei2001
model: routers
resolve_dns: false
interval: 1800
debug: false
run_once: false
threads: 30
use_max_threads: false
timeout: 20
timelimit: 300
retries: 3
prompt: !ruby/regexp /^([\w.@()-]+[#>]\s?)$/
next_adds_job: false
vars: {}
groups: {}
group_map: {}
models: {}
pid: "/root/.config/oxidized/pid"

extensions:
  oxidized-web:
    load: true
    listen: 0.0.0.0
    port: 8888

crash:
  directory: "/root/.config/oxidized/crashes"
  hostnames: false

stats:
stats:
  history_size: 10

input:
  default: ssh
  debug: false
  ssh:
    secure: false
  ftp:
    passive: true
  utf8_encoded: true

output:
  default: git
  git:
    user: Oxidized
    email: mahoe.dnl91@gmail.com
```

```
repo: "/root/.config/oxidized/configs.git"
single_repo: true
```

source:

default: csv

csv:

file: /root/.config/oxidized/router.db

delimiter: !ruby/regexp /:/

map:

name: 0

model: 1

username: 2

password: 3

model_map:

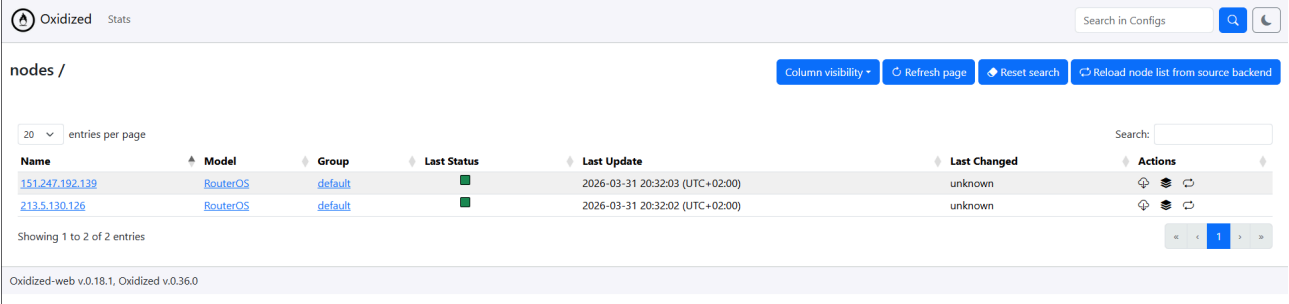
mikrotik: routeros

routeros: routeros

Contenu du fichier router.db :

213.5.130.126:routeros:admin:!&6GPJnt7DdwyNQws

151.247.192.139:routeros:admin:cloudfives



The screenshot shows the Oxidized web interface. At the top, there's a header with the Oxidized logo, 'Stats' link, and a search bar. Below the header, there's a 'nodes /' section with several action buttons: 'Column visibility', 'Refresh page', 'Reset search', and 'Reload node list from source backend'. A table lists two nodes. The table has columns for Name, Model, Group, Last Status, Last Update, Last Changed, and Actions. The first node is '151.247.192.139' with Model 'RouterOS', Group 'default', Last Status 'online', Last Update '2026-03-31 20:32:03 (UTC+02:00)', Last Changed 'unknown', and Actions 'edit', 'delete', 'refresh'. The second node is '213.5.130.126' with Model 'RouterOS', Group 'default', Last Status 'online', Last Update '2026-03-31 20:32:02 (UTC+02:00)', Last Changed 'unknown', and Actions 'edit', 'delete', 'refresh'. Below the table, it says 'Showing 1 to 2 of 2 entries'. At the bottom, there's a footer with 'Oxidized-web v0.18.1, Oxidized v0.36.0'.

Name	Model	Group	Last Status	Last Update	Last Changed	Actions
151.247.192.139	RouterOS	default	online	2026-03-31 20:32:03 (UTC+02:00)	unknown	edit delete refresh
213.5.130.126	RouterOS	default	online	2026-03-31 20:32:02 (UTC+02:00)	unknown	edit delete refresh

Showing 1 to 2 of 2 entries

Oxidized-web v0.18.1, Oxidized v0.36.0

[Figure 10 : Capture de l'interface Web Oxidized montrant la liste des équipements et les dates de sauvegarde]

6. Alerting et Dashboards

A - Création du Dashboard

1. Panel de l'Hyperviseur Proxmox (Agent Zabbix)

L'agent Zabbix 2 installé sur l'hôte Proxmox remonte des métriques système critiques :

Uptime : Permet de surveiller la stabilité de l'hôte et de détecter des redémarrages inattendus.

CPU Temperature & Utilization : Crucial pour prévenir la surchauffe du serveur physique (Thermal Throttling) et identifier les VM qui consomment trop de ressources.

Memory (RAM) & Storage : Surveillance de la saturation de la mémoire vive et de l'espace disque (LVM/ZFS) pour éviter le blocage de l'écriture des logs ou le crash des VM.

Disk Read/Write : Analyse des I/O (entrées/sorties) pour détecter d'éventuels goulots d'étranglement sur les disques physiques.

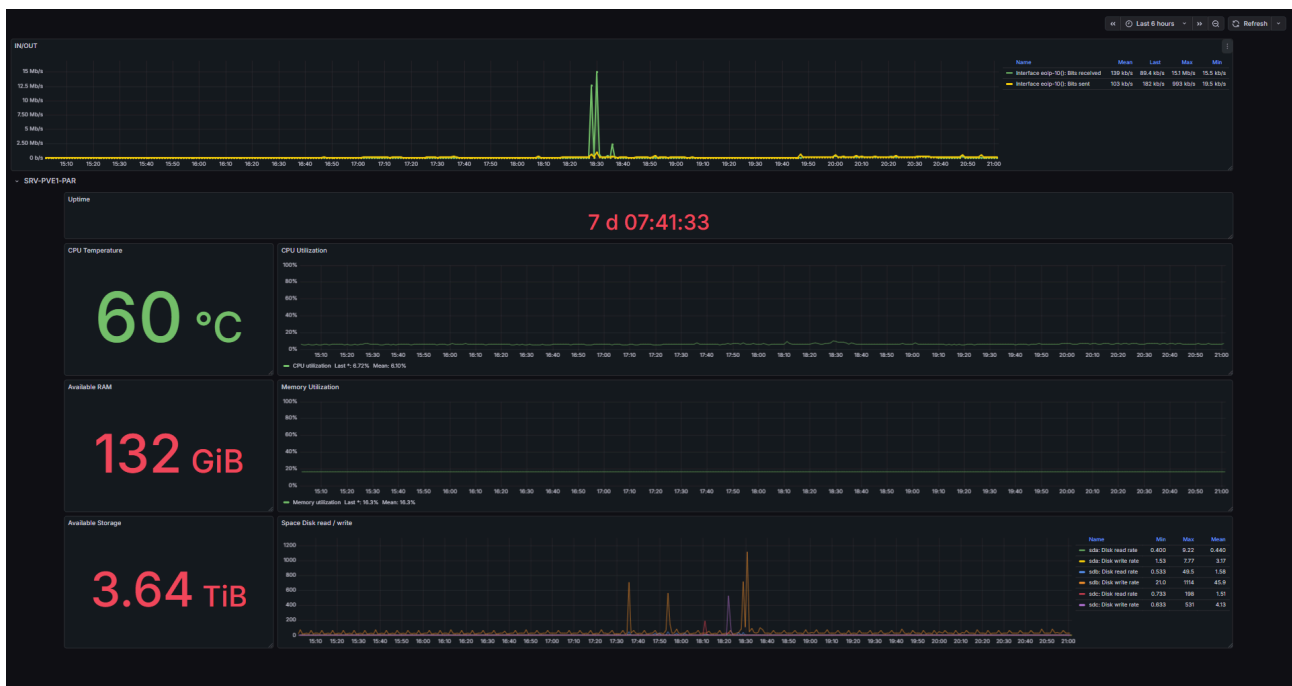
2. Panel du Tunnel EoIP (SNMP)

Le tunnel EoIP est le lien vital entre vos deux sites. Nous surveillons :

Bit send (Core01-PAR) : Le débit sortant depuis votre site local vers OVH.

Bit send (Core01-OVH) : Le débit sortant depuis OVH vers votre site local.

Analyse : La comparaison de ces deux graphiques permet de vérifier la symétrie du trafic et de s'assurer que le tunnel encapsule correctement les trames de niveau 2 sans perte de paquets.



[Figure 11 : Capture d'écran du Dashboard final combinant PVE et MikroTik]

B - Configuration des Alertes Critiques et Triggers

1. Alerte de rupture du Tunnel (Flux réseau)

Condition : Si Bits received de Core01-PAR est égal à 0.

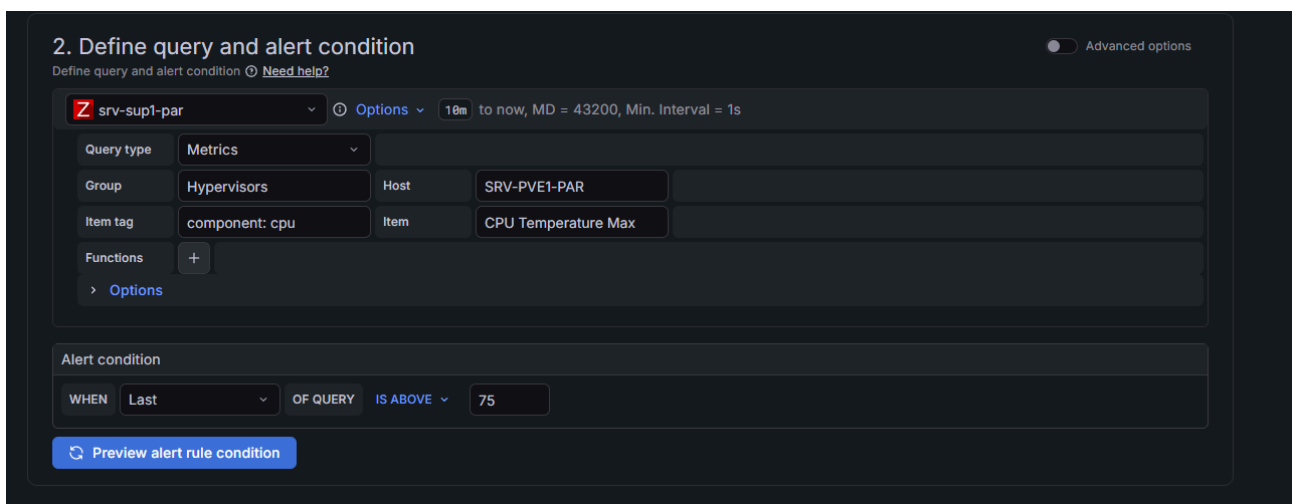
Logique : Si le trafic entrant sur l'interface EoIP tombe à zéro alors que le tunnel est censé être actif, cela signifie une rupture du lien

Action Grafana : Le panneau passe en fond rouge avec l'étiquette "**CRITICAL : TUNNEL DOWN**".

2. Alerte thermique (Sécurité matérielle)

Condition : Température CPU > **75°C**.

Logique : Les processeurs de serveurs ont des seuils de sécurité. À 75°C, on approche de la zone critique où les performances diminuent pour protéger le matériel. Une notification préventive permet d'intervenir sur la climatisation ou de réduire la charge des VM.



The screenshot shows the Grafana alert configuration page titled "2. Define query and alert condition". The interface is in a dark theme. At the top right, there is a toggle for "Advanced options". Below the title, there is a link for "Need help?". The main configuration area is divided into two sections: "Query" and "Alert condition".

Query Section:

- Query type: Metrics
- Group: Hypervisors
- Host: SRV-PVE1-PAR
- Item tag: component: cpu
- Item: CPU Temperature Max
- Functions: +
- Options: > Options

Alert condition Section:

- WHEN: Last
- OF QUERY: IS ABOVE
- Value: 75

At the bottom of the alert condition section, there is a button labeled "Preview alert rule condition".

[Figure 12 : Capture d'écran de la configuration de l'alerte]

C - Notification via Contact Points (Bot Telegram)

1. **Création du Bot** : Utilisation du "BotFather" sur Telegram pour générer un **Token API** unique.

2. **Configuration du Contact Point** :

Dans Grafana (**Alerting > Contact Points**), j'ai créé un nouveau point de contact de type **Telegram**.

Renseignement du **Chat ID** (mon identifiant utilisateur) et du **Token du Bot**.

3. **Fonctionnement** : Dès qu'un Trigger Zabbix passe à l'état "Problem" (ex: Température > 75°C), Grafana envoie une requête HTTP POST à l'API Telegram. Le bot m'envoie alors instantanément un message contenant le nom de l'alerte, la valeur actuelle et l'heure du déclenchement.



[Figure 13 : Capture d'écran de l'alerte reçue sur le téléphone via l'application Telegram]

III. Conclusion

Ce projet a permis de mettre en œuvre une infrastructure de supervision et de gestion de configuration complète pour un environnement hybride. Cette réalisation a requis l'utilisation de compétences avancées en termes de réseaux, de systèmes virtualisés et de développement de solutions de monitoring :

Administration de systèmes Linux : Installation et sécurisation de serveurs sous Ubuntu 24.04 et Debian 12 ;

Virtualisation de niveau professionnel : Configuration et optimisation de l'hyperviseur Proxmox VE 9.1 ;

Ingénierie Réseau : Mise en place d'un tunnel de niveau 2 (EoIP) entre un site local et un datacenter distant (OVH) ;

Sécurisation des flux : Configuration de règles de filtrage et de translation d'adresses via IPTables ;

Supervision et Métrologie : Déploiement de la stack Zabbix 7.0 avec remontée de données par agents et protocole SNMP ;

Analyse de données : Conception de tableaux de bord dynamiques sous Grafana pour l'aide à la décision ;

Automatisation de l'alerting : Interconnexion entre un système de supervision et l'API Telegram pour une réactivité mobile ;

Gestion de configuration (DevOps) : Mise en place d'Oxidized pour l'archivage et le versioning des configurations MikroTik.

Ce projet constitue une base solide pour la maintenance proactive de l'infrastructure de **DC2Scale**. Il permet de garantir une haute disponibilité des services et une traçabilité totale des modifications réseaux.

Je vous remercie pour votre attention.